```
const { createClient } = require('@libsql/client');
const crypto = require('crypto');

// Setup a safe local file database client for cross-platform support
const db = createClient({
  url: 'file:healthcare.db',
});

function hashPassword(password, salt) {
  return crypto.pbkdf2Sync(password, salt, 1000, 64, 'sha512').toString('hex');
}

function generateSalt() {
  return crypto.randomBytes(16).toString('hex');
}

async function initializeDatabase() {
  try {
    // Create users table
    await db.execute(`
      CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT UNIQUE NOT NULL,
        password_hash TEXT NOT NULL,
        salt TEXT NOT NULL,
        role TEXT NOT NULL CHECK(role IN ('Admin', 'Staff', 'Patient')),
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
      )
    `);

    // Create appointments table
    await db.execute(`
      CREATE TABLE IF NOT EXISTS appointments (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER,
        patient_name TEXT NOT NULL,
        email TEXT NOT NULL,
        phone TEXT NOT NULL,
        appointment_date TEXT NOT NULL,
        appointment_time TEXT NOT NULL,
        status TEXT DEFAULT 'pending',
        notes TEXT,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
      )
    `);

    // Create news table
    await db.execute(`
      CREATE TABLE IF NOT EXISTS news (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        content TEXT NOT NULL,
        image_url TEXT,
        category TEXT NOT NULL,
        date_posted DATETIME DEFAULT CURRENT_TIMESTAMP
      )
    `);

    // Seed default users if the table is empty
    const userCountRes = await db.execute("SELECT COUNT(*) as count FROM users");
    const userCount = userCountRes.rows[0].count;
    if (userCount === 0) {
      const adminSalt = generateSalt();
      const staffSalt = generateSalt();
      const patientSalt = generateSalt();

      await db.execute({
        sql: "INSERT INTO users (username, email, password_hash, salt, role) VALUES (?,
```

```
          ?, ?, ?, ?)",
          args: ["admin", "admin@alamnagar-chc.org", hashPassword("adminpass", adminSalt),
          adminSalt, "Admin"]
        });
        await db.execute({
          sql: "INSERT INTO users (username, email, password_hash, salt, role) VALUES (?,
          ?, ?, ?, ?)",
          args: ["staff", "staff@alamnagar-chc.org", hashPassword("staffpass", staffSalt),
          staffSalt, "Staff"]
        });
        await db.execute({
          sql: "INSERT INTO users (username, email, password_hash, salt, role) VALUES (?,
          ?, ?, ?, ?)",
          args: ["patient", "patient@example.com", hashPassword("patientpass",
          patientSalt), patientSalt, "Patient"]
        });
        console.log("Seeded default users (admin, staff, patient).");
      }

      // Insert initial news items if table is empty
      const newsCountRes = await db.execute("SELECT COUNT(*) as count FROM news");
      const newsCount = newsCountRes.rows[0].count;
      if (newsCount === 0) {
        await db.execute({
          sql: "INSERT INTO news (title, content, image_url, category) VALUES (?, ?, ?,
          ?)",
          args: [
            "Free Medical Health Camp Next Saturday",
            "Alamnagar Charitable Healthcare Centre is organizing a free health check-up
            camp next Saturday. General physicians, pediatricians, and cardiologists will
            be available for consultations from 9:00 AM to 3:00 PM. Free medicine
            distribution is also arranged.",
            "https://unsplash.com",
            "Event"
          ]
        });
        await db.execute({
          sql: "INSERT INTO news (title, content, image_url, category) VALUES (?, ?, ?,
          ?)",
          args: [
            "New Pediatric Specialist Joins Our Team",
            "We are pleased to welcome Dr. Sarah Rahman, MD in Pediatrics, to our medical
            team. She will be available for consultations every Monday and Wednesday
            starting next week. Book your appointments online.",
            "https://unsplash.com",
            "News"
          ]
        });
        console.log("Inserted initial news items.");
      }
    } catch (err) {
      console.error('Database initialization error:', err.message);
    }
  }

  // Initialize database
  initializeDatabase();

  // Database query helpers matching async format
  module.exports = {
    hashPassword,
    generateSalt,

    getUserByUsername: async (username) => {
      const res = await db.execute({ sql: "SELECT * FROM users WHERE username = ?", args:
      [username] });
      return res.rows[0] || null;
    },

```

```
125     getUserByEmail: async (email) => {
126       const res = await db.execute({ sql: "SELECT * FROM users WHERE email = ?", args:
          [email] });
127       return res.rows[0] || null;
128     },
129
130     createUser: async (user) => {
131       const { username, email, password, role } = user;
132       const salt = generateSalt();
133       const hash = hashPassword(password, salt);
134       const query = `INSERT INTO users (username, email, password_hash, salt, role) VALUES
          (?, ?, ?, ?, ?)`;
135       const res = await db.execute({ sql: query, args: [username, email, hash, salt, role]
          });
136       return { id: Number(res.lastInsertRowid), username, email, role };
137     },
138
139     getAllAppointments: async () => {
140       const res = await db.execute("SELECT * FROM appointments ORDER BY appointment_date
          ASC, appointment_time ASC");
141       return res.rows;
142     },
143
144     getAppointmentsByUserId: async (userId) => {
145       const res = await db.execute({ sql: "SELECT * FROM appointments WHERE user_id = ?
          ORDER BY appointment_date ASC, appointment_time ASC", args: [userId] });
146       return res.rows;
147     },
148
149     createAppointment: async (appointment) => {
150       const { user_id, patient_name, email, phone, appointment_date, appointment_time,
          notes } = appointment;
151       const query = `INSERT INTO appointments (user_id, patient_name, email, phone,
          appointment_date, appointment_time, notes) VALUES (?, ?, ?, ?, ?, ?, ?)`;
152       const res = await db.execute({ sql: query, args: [user_id || null, patient_name,
          email, phone, appointment_date, appointment_time, notes || ""] });
153       return { id: Number(res.lastInsertRowid), ...appointment, status: 'pending' };
154     },
155
156     updateAppointmentStatus: async (id, status) => {
157       const query = `UPDATE appointments SET status = ? WHERE id = ?`;
158       const res = await db.execute({ sql: query, args: [status, id] });
159       return { changes: res.rowsAffected };
160     },
161
162     getAllNews: async () => {
163       const res = await db.execute("SELECT * FROM news ORDER BY date_posted DESC");
164       return res.rows;
165     },
166
167     createNews: async (newsItem) => {
168       const { title, content, image_url, category } = newsItem;
169       const query = `INSERT INTO news (title, content, image_url, category) VALUES (?, ?,
          ?, ?)`;
170       const res = await db.execute({ sql: query, args: [title, content, image_url || "",
          category] });
171       return { id: Number(res.lastInsertRowid), ...newsItem, date_posted: new
          Date().toISOString() };
172     }
173   };
174
```